

HyQuant: Hybrid-Precision Quantization for LLM Attention

Jiatong Ding^{1*}, Bingxin Xing^{2*}, Yu Zhang⁴, Dian Ding^{1†}, Xiaodong Yi³, Xianbin Ouyang³, Feihu Zhou³, Kun Zhang³, Zhenyu Guo³, Hao Pan¹, Guangtao Xue¹, Yiming Zhang¹

¹Shanghai Jiao Tong University, ²Xi'an Jiaotong University, ³Tencent Penglai Lab, ⁴Xiamen University

Abstract

Quantization has been widely adopted in LLM training and inference to reduce cost and improve efficiency. However, low-bit quantization of the *attention* module often introduces large errors at very low bit-widths, causing performance degradation. Existing methods mainly rely on smoothing techniques to handle outliers, while we propose a hybrid quantization design to better balance accuracy and efficiency. Specifically, we propose **HyQuant**, an efficient hybrid quantization framework for LLM attention. HyQuant quantizes most attention states into low-bit formats while retaining a small set of vertical-line tokens and local-window states in high precision. These accuracy-critical regions are selected using lightweight vertical-line-aware attention-pattern signals, reducing quantization error with limited overhead. In the Prefill stage, HyQuant uses a hybrid-precision quantized attention operator that preserves vertical-line tokens and a local sliding window in full precision while quantizing the remaining context. In the Decode stage, HyQuant applies the same principle to KV-cache compression and fuses KV dequantization with attention computation to improve memory and hardware efficiency. Across diverse tasks, models, and datasets, HyQuant maintains nearly lossless accuracy with an extremely simple design, demonstrating the efficiency and practical feasibility of hybrid quantization for LLM attention. Code is available at: <https://github.com/jerrysfls/HyQuant>.

1 Introduction

With OpenAI’s o1 series (OpenAI, 2024), Gemini (Gemini Team, 2025), and DeepSeek (DeepSeek-AI, 2025) bringing Chain-of-Thought (CoT) reasoning into mainstream, Large Language Models (LLMs) have shifted from brief answers to long multi-step

reasoning traces, often reaching tens of thousands of tokens.

In long-context inference, bottlenecks differ across stages. During *Prefill*, full-prefix attention is dominated by large matrix multiplications and is mainly compute-bound. During *Decode*, each new token repeatedly reads/writes past KV states, so memory capacity and bandwidth become the constraints.

Quantization is widely deployed to reduce compute and memory costs, but aggressively pushing precision to very low bit-widths can degrade end-to-end model quality (Zheng et al., 2026). In our setting, uniform token-wise precision further fails to account for the non-uniform sensitivity induced by imbalanced attention. This stems from *non-uniform sensitivity* under imbalanced attention, a few high-contribution positions dominate the error, so uniform compression over-compresses critical tokens while wasting budget elsewhere. Existing approaches, whether sparsification-based (eviction, block-sparse attention) or quantization-based (KV-cache quantization, approximated attention) largely treat tokens as homogeneous and **do not answer a reasoning-centric mixed-precision question**: how to design a mixed precision that accounts for heterogeneous token sensitivity to balance accuracy and efficiency in long-context CoT inference.

As shown in Fig. 1, we repeatedly observe *vertical-line* structures across attention heatmaps, consistent with prior observations (Jiang et al., 2024). Unlike drifting diagonal and oblique bands, these vertical lines carry higher and more stable attention mass, but cover only a tiny fraction of tokens (typically < 5%). This motivates preserving such positions in full precision during low-bit quantization.

Motivated by these observations, we propose **HyQuant**, a **mixed-precision quantization** method that keeps full precision for a tiny set of positions—persistent *vertical-line* positions and a

* Equal contribution. † Corresponding author.

recent *sliding window* while quantizing the rest to low bit-width. HyQuant identifies vertical lines with a lightweight procedure and uses a fixed window size with small overhead. In **Prefill** (compute-intensive), we design a quantized attention operator that runs most computation in low precision while keeping vertical lines and the local window in full precision, fusing both paths into a single operator. In **Decode** (memory-intensive), we quantize the KV cache to reduce capacity and bandwidth pressure while keeping the same critical positions unquantized, and fuse KV dequantization with attention computation for efficiency.

We evaluate **HyQuant** on Qwen3-8B, Qwen3-32B, LLaMA3.1-8B, and GLM-4-9B with LongBench, GSM8K, and MATH500, achieving $1.32\times$ to $3.58\times$ decode-kernel speedup and $1.04\times$ to $1.17\times$ end-to-end decode speedup, while maintaining near-full-precision accuracy and improving over strict low-bit baselines in most settings.

2 Motivation

2.1 A Small Number of Tokens Remain Persistently Important

HyQuant is motivated by a recurring observation in real inference: LLM attention is highly non-uniform, with a large fraction of attention mass concentrated on a small set of salient tokens. Similar structures have been reported in prior long-context studies, including the vertical-line and vertical-slash patterns in MInference (Jiang et al., 2024) and the attention sink phenomenon in StreamingLLM (Xiao et al., 2024). Although these patterns differ in form, they reveal the same underlying property: only a small subset of key/value positions are repeatedly attended to across long generation spans. These positions act as persistent attention anchors. Depending on the pattern, they may correspond either to sink-like special positions or to semantically relevant tokens that are repeatedly referenced by later queries.

Fig. 1 shows that this vertical-stripe pattern is pervasive within Qwen3-8B. Beyond the local diagonal structure, many layers and heads exhibit bright vertical columns that are attended by many query tokens, indicating that the *effective attention support* is concentrated rather than uniformly distributed over the full sequence. The same figure further shows that this phenomenon is not limited to Qwen3-8B: recent model families, including Gemma4-31B, Qwen3.5-4B, and Llama3-8B, ex-

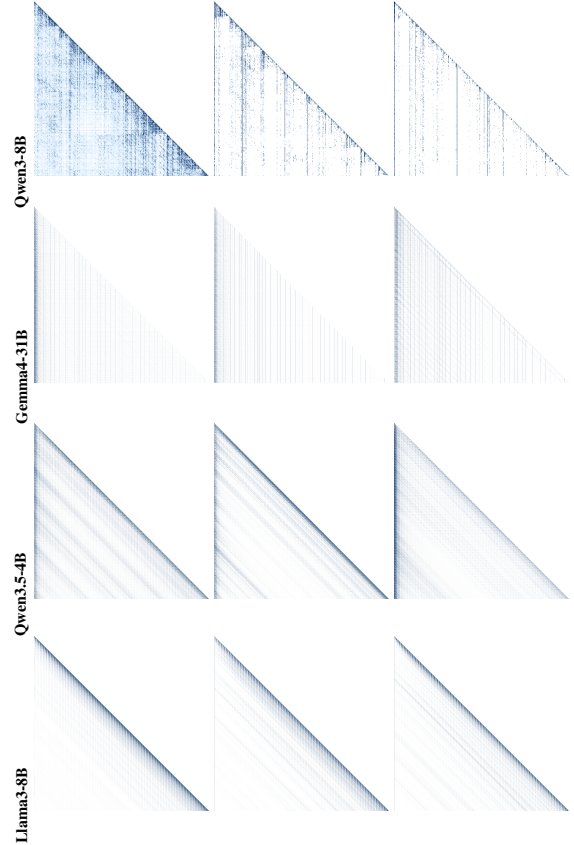


Figure 1: **Vertical-line-dominated attention across model families.** Rows from top to bottom: Qwen3-8B, Gemma4-31B, Qwen3.5-4B, and Llama3-8B. Across layers, heads, and model families, persistent vertical bright stripes indicate that a small number of positions are repeatedly attended over long spans, revealing a highly imbalanced attention distribution and motivating compression strategies that prioritize a small set of critical positions.

hibit similar sink-like vertical patterns.

This observation is consistent with prior analyses of attention sinks (Xiao et al., 2024; Gu et al., 2025). Recent gated-attention mechanisms have been proposed to mitigate such sink effects (Qiu et al., 2025), but our heatmaps suggest that sink-like vertical concentration is still not fully removed in modern models. Therefore, token-agnostic KV eviction or uniform low-bit quantization can be mismatched with real long-context inference dynamics, motivating hybrid compression methods that explicitly preserve these persistently attended tokens.

Evidence: a small set of high-score positions covers most attention mass. We further quantify this concentrated-attention phenomenon on **Llama-3.1-8B** and **Qwen3-8B**. For each model, we measure the fraction of attention mass, computed from

Table 1: **Attention-mass coverage across model families.** We report the fraction of attention mass covered by the global top-1% key positions (TOP-1%), the global top-5% key positions (TOP-5%), and the union of the global top-5% key positions and a local window $W=128$ (TOP-5%+WIN).

Model	TOP-1%	TOP-5%	TOP-5%+WIN
Llama-3.1-8B	58.83%	64.09%	85.63%
Qwen3-8B	47.30%	54.10%	82.53%

$\text{softmax}(QK^\top)$, covered by: (i) the global top-1% highest-scoring key positions (TOP-1%), (ii) the global top-5% highest-scoring key positions (TOP-5%), and (iii) the union of the global top-5% key positions and a local window $W=128$ (TOP-5%+WIN). As shown in Table 1, a small fraction of high-score key positions already covers a substantial portion of the total attention mass, and combining them with a short local window further increases coverage to over 80%. This confirms that the effective attention support is dominated by a small set of persistently important tokens together with recent local context, motivating a *non-uniform* precision allocation that aligns the precision budget with this concentrated structure.

2.2 Quantization Errors Are Amplified at High-Score Positions

Low-bit quantization errors are not equally harmful across tokens: they are most destructive at *high-score* positions, where small K/V perturbations get amplified through repeated KV access by many queries across prefill and decode. As a result, uniform quantization with a fixed compression rate often fails to retain a small number of critical tokens in sufficient precision, leading to considerable quality degradation that is consistent with the attention imbalance observed in Section 2.1.

Evidence: retaining a tiny set of high-score positions in full precision substantially reduces error. We conduct error analysis on Qwen3-8B and use full-precision FlashAttention as the reference, measuring the MSE of the intermediate attention output (input to `o_proj`) under different quantization strategies. As shown in Fig. 2, uniform 4-bit quantization yields noticeably larger errors than uniform 8-bit quantization. More importantly, when the remaining positions are quantized to 4-bit, retaining only the top-1%/top-5% high-score positions in full precision dramatically suppresses

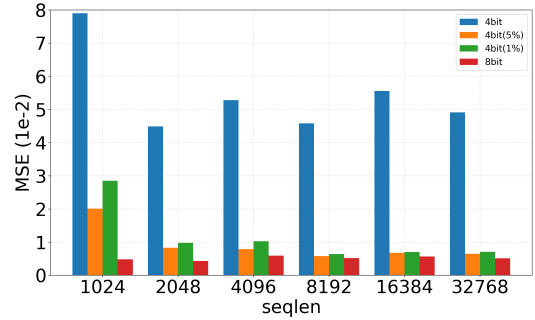


Figure 2: **Amplified quantization error at high-score positions (Qwen3-8B, Layer 28).** Uniform 4-bit quantization incurs substantially larger MSE than uniform 8-bit quantization. When the remaining positions are quantized to 4-bit, retaining only the top-1%/top-5% high-score positions in full precision significantly reduces the error and consistently approaches the 8-bit error level across different sequence lengths.

the error—consistently approaching the 8-bit error level across sequence lengths from 1K to 32K. This directly motivates the hybrid-precision design of HyQuant: with a small full-precision budget, HyQuant prioritizes critical high-attention K/V positions while quantizing the remaining majority to low bit-width.

3 Related Work

3.1 Sparse and Selective Long-Context Inference

A major line of work accelerates long-context inference by skipping low-contribution attention blocks or selectively retaining cache content. For *Prefill*, block-sparse attention reduces FLOPs by computing only selected blocks, often relying on online scoring, retrieval, or reordering. These methods are effective at very long contexts, but their pre-processing overhead can offset the saved FLOPs at short or moderate sequence lengths. For *Decode*, KV-cache eviction and selective retention methods such as H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2025), and HeadKV (Fu et al., 2025) reduce memory footprint. However, eviction can be brittle for long CoT reasoning, where token importance is non-stationary and previously removed content may later become critical. In contrast, the vertical-line phenomenon we exploit is measured at the query-distribution level: certain key positions receive high attention from a large fraction of query positions rather than from only a single query. Similar persistent concentration patterns have been reported in analyses of

dynamic sparse attention and attention sinks (Jiang et al., 2024; Xiao et al., 2024).

3.2 Low-Bit Attention and KV-Cache Quantization

Quantization reduces bandwidth and storage via low-bit representations. In *Prefill*, quantized or approximated attention methods such as SageAttention (Zhang et al., 2025) improve attention efficiency, but aggressive bit-widths can introduce noticeable quality loss. In *Decode*, KV-cache quantization is more mature: KIVI (Liu et al., 2024), KV-Tuner (Li et al., 2025), and KVQuant (Hooper et al., 2024) reduce quantization error through asymmetric quantization, sensitivity-aware allocation, or outlier handling. These methods mainly focus on reducing quantization error under compact representations, but they do not explicitly exploit persistent vertical-line structures for token-level full-precision retention in long-context reasoning.

3.3 Difference from previous work.

A closely related work, exemplified by MInference (Jiang et al., 2024), also identifies vertical-line structures in attention maps. However, unlike MInference, which employs these patterns as a sparsity mask, we use them to assign different precisions to tokens of various importance. This method retains information from less significant tokens in a relatively low bit form, which is totally overlooked in MInference. To visualize the effect in long-context settings, we conduct an ablation study on Qwen3-8B and Longbench dataset, whose results can be seen in Section B.6. In addition, the idea of sensitivity-aware mixed-precision appeared in KV-Tuner (Li et al., 2025), yet the author applied it to different layers rather than tokens in our work.

Another highlight in our work is that we propose an integrated Prefill-Decode method, providing optimization to both ends, while previous methods usually deal with only one stage. According to identified vertical line tokens, we implement operand quantization in prefill stage, and both operand and cache quantization in decode stage. This results in both accelerated speed, decreased memory usage, and increased accuracy.

4 Method

We present HyQuant, a vertical-line-aware hybrid-precision quantization framework for long-context inference. As illustrated in Fig. 1, a tiny fraction of

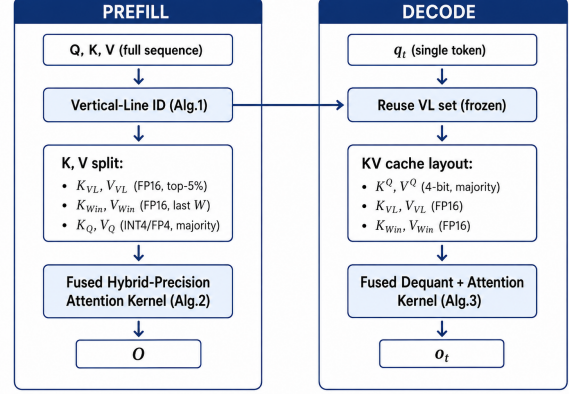


Figure 3: **Overview of HyQuant.** HyQuant retains vertical-line positions and a local window in full precision, while quantizing the remaining majority for efficient prefill and decode.

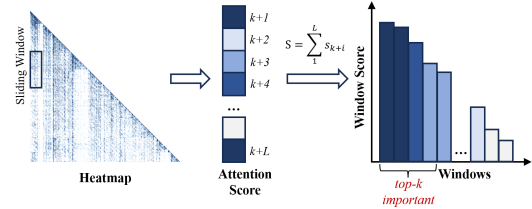


Figure 4: **Vertical-line awareness process.** HyQuant estimates column-wise attention mass to identify persistent vertical-line positions for full-precision retention.

tokens (typically $< 5\%$) forms persistent *vertical lines* that carry disproportionately large attention mass and dominate quantization error under aggressive low-bit settings. HyQuant retains these error-sensitive vertical-line positions and a local window in full precision, while quantizing the remaining majority to low bits. Concretely, HyQuant consists of three components: (i) **vertical-line-aware high-precision retention**, (ii) **Prefill-stage hybrid-precision quantized attention**, and (iii) **Decode-stage hybrid low-bit KV cache with fused attention**.

4.1 Vertical-line Awareness

Token sets. For each layer/head, we partition the key positions into three disjoint subsets:

$$\mathcal{K} = \mathcal{K}_{VL} \cup \mathcal{K}_{Win} \cup \mathcal{K}_Q, \quad (1)$$

where $\mathcal{K}_{VL}$ denotes the *vertical-line* positions, $\mathcal{K}_{Win}$ denotes a fixed recent sliding window, and $\mathcal{K}_Q$ denotes the remaining majority to be quantized. Vertical-line positions are selected from the non-window prefix, making the three subsets disjoint by construction.

Window definition. Given the current query index t , the window keys are

$$\mathcal{K}_{\text{Win}}(t) = \{k \mid k \in [\max(0, t-W+1), t]\}, \quad (2)$$

where W is a pre-specified window size. We denote the non-window prefix by

$$\mathcal{K}_{\text{pre}}(t) = \mathcal{K} \setminus \mathcal{K}_{\text{Win}}(t). \quad (3)$$

Vertical-line identification. We maintain a running importance score for each non-window key position that measures the accumulated *column mass* (vertical attention mass). Let $a_{t,k}$ be the attention probability from query t to key k . We define the vertical-line score as

$$S(k) = \sum_{t \in \mathcal{T}} a_{t,k}, \quad (4)$$

where $\mathcal{T}$ is the set of queries considered (e.g., within the current segment or within a running buffer). We then select the top- ρ fraction from the non-window prefix as vertical-line positions:

$$\mathcal{K}_{\text{VL}} = \text{TopK}(S|_{\mathcal{K}_{\text{pre}}}, \rho \cdot |\mathcal{K}_{\text{pre}}|), \quad \rho \ll 1. \quad (5)$$

In practice, ρ is small. Importantly, $S(k)$ can be updated with negligible overhead using a lightweight reduction, and the window size W is fixed in advance; therefore, **vertical-line awareness introduces virtually no extra preprocessing cost** compared to sparsification pipelines. The detailed algorithm is shown in Algorithm 1.

4.2 Prefill-stage Hybrid-Precision Quantized Attention

Hybrid-precision quantized attention in Prefill. In Prefill, attention is compute-intensive and dominated by large GEMMs. HyQuant quantizes the majority of attention computation, while retaining full precision on $\mathcal{K}_{\text{VL}}$ and $\mathcal{K}_{\text{Win}}$.

Specifically, for a query block Q , we conceptually split keys/values into

$$(K, V) = (K_Q, V_Q) \cup (K_{\text{FP}}, V_{\text{FP}}), \quad (6)$$

where (K_Q, V_Q) correspond to $\mathcal{K}_Q$, and $(K_{\text{FP}}, V_{\text{FP}})$ correspond to $\mathcal{K}_{\text{VL}} \cup \mathcal{K}_{\text{Win}}$.

We compute attention with a fused operator:

$$O = \text{Softmax}\left(\frac{Q\hat{K}_Q^\top}{\sqrt{d}} \parallel \frac{QK_{\text{FP}}^\top}{\sqrt{d}}\right) \cdot (\hat{V}_Q \parallel V_{\text{FP}}), \quad (7)$$

where $\hat{K}_Q, \hat{V}_Q$ are dequantized values of low-bit (K_Q, V_Q) , and $\parallel$ denotes concatenation along the key dimension. The concatenation order follows the physical KV layout used by the fused kernel. Crucially, we *fuse* the full-precision path and the quantized path into a single kernel so that the operator structure remains FlashAttention-like (online softmax with blockwise scanning), and the additional overhead over naive low-bit attention quantization is minimal.

Implementation note. In our implementation, we use lower precision for the bulk computation (e.g., INT8/FP8, INT4/FP4 depending on the backend), while retaining the vertical-line and window tiles in FP16/BF16. This hybrid-precision execution preserves the accuracy-critical attention mass with a tiny full-precision budget. The detailed algorithm is shown in Algorithm 2.

4.3 Decode-stage Low-bit KV Cache and Fused Attention

Low-bit KV cache with selective full precision. Decode is memory-intensive: each step reads a large prefix KV. HyQuant stores the KV cache in a hybrid-precision manner:

$$(K_{1:t}, V_{1:t}) = (K_{1:t}^Q, V_{1:t}^Q) \cup (K_{1:t}^{\text{FP}}, V_{1:t}^{\text{FP}}), \quad (8)$$

where full precision is reserved for keys in $\mathcal{K}_{\text{VL}} \cup \mathcal{K}_{\text{Win}}(t)$, and the remaining majority is stored in low bits. This directly reduces KV memory footprint and bandwidth pressure.

Fused dequantization and attention. To avoid extra memory traffic, we fuse dequantization into the decode attention kernel: when scanning a quantized KV block, we dequantize on the fly and immediately apply the values to score and value accumulation using FlashAttention-style online softmax. Formally, for a quantized block (K_b^Q, V_b^Q) ,

$$\hat{K}_b = \text{DeQuant}(K_b^Q), \hat{V}_b = \text{DeQuant}(V_b^Q), \quad (9)$$

and the kernel updates the running softmax state and output accumulator using $(\hat{K}_b, \hat{V}_b)$ without materializing full-precision intermediates. For blocks belonging to $\mathcal{K}_{\text{VL}} \cup \mathcal{K}_{\text{Win}}(t)$, we directly load full-precision keys/values.

Summary. By retaining only a tiny set of vertical-line positions and a local window in full precision, HyQuant controls the dominant quantization errors, while quantizing the remaining majority to achieve

low-bit efficiency. In Prefill, HyQuant preserves low-bit attention efficiency while reducing numerical error; in Decode, it reduces KV memory traffic together with fused dequantization for bandwidth efficiency. The detailed algorithm is shown in Algorithm 3.

5 Experiments

5.1 Experimental Setup

Models. We primarily evaluate HyQuant on **Qwen3-8B** (Yang et al., 2025) and further validate its generality on **Llama-3.1-8B-Instruct** (Grattafiori et al., 2024), **GLM-4-9B-0414** (GLM Team, 2024), and **Qwen3-32B** (Yang et al., 2025). We focus on long-context reasoning capability and adopt predominantly chain-of-thought task settings to cover multi-step reasoning and information aggregation under long sequences.

Baselines. We compare against representative full-precision and low-bit inference baselines: (i) **FlashAttention-2 (FA2)** (Dao, 2024), which serves as the full-precision accuracy and latency baseline; (ii) **KIVI** (Liu et al., 2024), a KV-cache quantization baseline for reducing memory and bandwidth overhead during long-context inference; (iii) **KV-Tuner** (Li et al., 2025), a sensitivity-aware mixed-precision KV-cache quantization baseline; and (iv) **SageAttention** (Zhang et al., 2025), an attention quantization baseline that mitigates activation outliers and uses a numerically stable computation path. For each model, we report the applicable baselines depending on implementation availability. Detailed settings can be seen in B.8.

Implementation Details (HyQuant). Unless specified otherwise, all experiments are conducted on an **NVIDIA H100** GPU. HyQuant adopts a hybrid-precision design for both Prefill attention and Decode KV-cache compression. We retain two categories of positions in full precision: (i) the online-identified top-5% vertical-line tokens, and (ii) tokens inside a local sliding window. All remaining KV positions are stored in Key-4bit, Value-4bit quantized formats. Quantized caches are recovered through dequantization and hybrid-precision computation to balance accuracy and efficiency.

Benchmarks and metrics. For accuracy, we evaluate mathematical reasoning on **GSM8K** (Cobbe et al., 2021) and **MATH500** (Hendrycks et al., 2021; Lightman

et al., 2023), and assess long-context capability on **LongBench v1** (Bai et al., 2024). For efficiency and operator-level analysis, we focus on the Prefill and Decode stages and report: (i) **Prefill operator-level error**: using full-precision FlashAttention-2 as the reference, we compute the layer-wise MSE of the intermediate attention output (i.e., the input to o_{proj}) to quantify numerical deviations introduced by low-bit computation, and compare against SageAttention; (ii) **Decode latency**: under the same decoding setup, we report the per-step total decoding latency (ms/token) across different prefix lengths and compare with FA2, together with the relative speedup. All latency numbers are collected after sufficient warmup and averaged over multiple runs.

5.2 Main Results

5.2.1 Benchmark Results

Tables 2, 3, 4, 5, and 9 summarize the end-to-end benchmark results. Across Qwen3-8B, Llama-3.1-8B-Instruct, GLM-4-9B-0414, and Qwen3-32B, HyQuant generally preserves the performance of full-precision attention (FA2) and improves over the applicable strict low-bit baselines, including KIVI, KV-Tuner, and SageAttention, in most settings. This indicates that a hybrid-precision strategy that *retains a small set of vertical-line tokens together with a local window in full precision* is effective for preserving long-context understanding and mathematical reasoning ability. For Qwen3-8B, we enable thinking mode in end-to-end evaluation to better reflect realistic long-CoT generation and KV reuse.

5.2.2 Operator-level evaluation: Prefill error and Decode latency

Prefill: layer-wise MSE. Since the Prefill latency of HyQuant is comparable to that of SageAttention under our implementation, we focus on numerical deviations rather than additional prefill speedup claims in this stage. We use full-precision FA2 as the reference and compute the **MSE** of the intermediate attention output (i.e., the input to o_{proj}) for each layer. We further report the **MSE reduction factor** of HyQuant over SageAttention, defined as $\text{MSE}_{\text{Sage}}/\text{MSE}_{\text{HyQuant}}$. As shown in Fig. 5, we visualize the first five layers as representative examples. HyQuant achieves a large reduction in MSE over SageAttention in these layers, demonstrating that *retaining vertical-line tokens and the local window in full precision* effectively

Table 2: **LongBench v1 results on Qwen3-8B (thinking mode)**. We compare full-precision attention (FA2), KIVI, KVtuner, SageAttention, and HyQuant (Ours). **Avg.** denotes the arithmetic mean over the 11 evaluated LongBench tasks reported in this table.

Method	Single-Document QA			Multi-Document QA			Summarization		Few-shot Learning		Semantic	Avg.
	NarrQA	Qasper	MF-en	HotpotQA	2Wiki	Musique	QMSum	MNews	Trivia	SAMSum	Pretrieve-en	
FA2 (full-precision)	30.37	41.68	50.34	50.42	39.69	33.88	13.11	10.85	97.50	33.80	88.81	44.59
KIVI (K4V4)	24.76	35.13	45.51	42.55	31.04	18.92	10.51	8.19	88.87	27.56	81.41	37.68
SageAttention	19.45	34.54	41.95	46.59	44.33	24.31	18.67	23.23	74.93	32.62	58.86	38.13
KVTuner (4bit)	13.74	32.45	45.32	37.68	54.18	31.92	20.20	10.67	94.70	29.32	74.81	40.45
HyQuant (K4V4, top-5%)	32.28	41.82	49.74	49.75	43.31	35.24	15.39	11.94	96.65	33.05	86.32	45.04

Table 3: **LongBench v1 results on Llama-3.1-8B-Instruct**. We compare full-precision attention (FA2), KVtuner, SageAttention, and HyQuant (Ours). **Avg.** denotes the reported average score over the evaluated LongBench tasks.

Method	Single-Document QA			Multi-Document QA			Summarization		Few-shot Learning		Semantic	Avg.
	NarrQA	Qasper	MF-en	HotpotQA	2Wiki	Musique	QMSum	MNews	Trivia	SAMSum	Pretrieve-en	
FA2 (full-precision)	30.96	45.16	54.60	55.63	45.70	31.69	24.26	26.17	88.36	12.48	97.90	46.63
SageAttention	31.16	43.29	55.01	51.81	46.97	30.99	24.96	26.64	86.67	6.87	98.15	45.68
KVTuner (4bit)	31.29	42.06	53.97	50.11	45.57	32.49	24.91	27.08	88.42	7.68	98.70	45.66
HyQuant (K4V4, top-5%)	30.75	46.51	55.84	51.22	45.30	31.90	24.84	27.21	87.67	13.75	99.12	46.73

Table 4: **LongBench v1 results on GLM-4-9B-0414**. We compare full-precision attention (FA2), SageAttention, KVtuner, and HyQuant (Ours). **Avg.** denotes the reported average score over the evaluated LongBench tasks.

Method	Single-Document QA			Multi-Document QA			Summarization		Few-shot Learning		Semantic	Avg.
	NarrQA	Qasper	MF-en	HotpotQA	2Wiki	Musique	QMSum	MNews	Trivia	SAMSum	Pretrieve-en	
FA2 (full-precision)	24.02	39.67	51.49	40.08	50.86	26.43	22.06	23.86	85.73	37.72	91.26	44.83
SageAttention	24.85	43.30	52.05	47.10	47.66	29.42	23.16	21.37	88.46	35.64	90.25	45.75
KVTuner (4bit)	23.51	41.39	50.14	46.65	46.57	32.16	23.05	24.25	85.92	37.14	91.15	45.63
HyQuant (K4V4, top-5%)	25.41	40.07	49.97	48.70	49.18	26.30	22.42	24.52	87.46	37.48	92.10	45.78

suppresses error amplification under low-bit computation.

Decode: decoding latency vs. FA2. In the Decode stage, we compare HyQuant against FA2 in terms of decode attention-kernel latency under different prefix lengths. Table 7 summarizes the kernel-level results: as the context grows, HyQuant achieves increasingly larger gains, reaching about $3.58\times$ speedup over FA2 at the 32K prefix length, while the corresponding end-to-end decode speedup is more moderate. This highlights the benefits of hybrid-precision KV storage and fused dequantization for bandwidth- and memory-bound attention kernels in long-context decoding.

5.3 High parallel setting speedup

While HyQuant achieves significant speedup under batch size 1 and maintains accuracy, its performance under high parallel, reality settings remains unknown. To demonstrate its ability, we compare HyQuant with Flashattention-2 (Dao, 2024), KIVI (Liu et al., 2024), KVtuner (Li et al., 2025) under multi-batch settings. Results can be seen in

6.

The results show that HyQuant is the only method that operates in batch size 32, showing its superiority, while outperforms other methods in terms of speed in lower batch settings. This is a mutual effect from KV cache compression and employing a fused kernel, decreasing bandwidth requirements and peak memory occupation. Compared with our implementations of KIVI (Liu et al., 2024) and KVtuner (Li et al., 2025), which follow a “dequantize-then-attend” strategy—the quantized KV cache is dequantized before running standard attention—HyQuant adopts a fused “dequantize-in-attention” design that unpacks the KV cache within a single online-softmax pass and never materializes the full-precision cache. The results demonstrate our method’s advantage, which employs an operator-level co-design of compression and computation.

Summary. In our experiments on Qwen3-8B, Llama-3.1-8B-Instruct, GLM-4-9B-0414, and Qwen3-32B, HyQuant achieves a favorable

Table 5: **LongBench v1 results on Qwen3-32B (thinking mode)**. We compare full-precision attention (FA2), KIVI, KVTuner, and HyQuant (Ours). **Avg.** denotes the reported average score over the evaluated LongBench tasks.

Method	Single-Document QA			Multi-Document QA			Summarization		Few-shot Learning		Semantic	Avg.
	NarrQA	Qasper	MF-en	HotpotQA	2Wiki	Musique	QMSum	MNews	Trivia	SAMSum	Pretrieve-en	
FA2 (full-precision)	34.93	46.38	43.79	54.18	50.57	45.59	17.11	14.74	90.60	36.80	100.00	48.61
KIVI	19.89	42.21	37.87	47.42	46.16	41.30	12.67	11.16	80.37	28.68	94.00	41.98
KVTuner (4bit)	32.97	45.64	44.89	52.13	50.47	45.05	16.79	14.82	90.60	37.10	100.00	48.22
HyQuant (K4V4, top-5%)	45.56	41.05	46.17	44.44	53.12	43.76	17.15	14.89	90.60	36.35	100.00	48.46

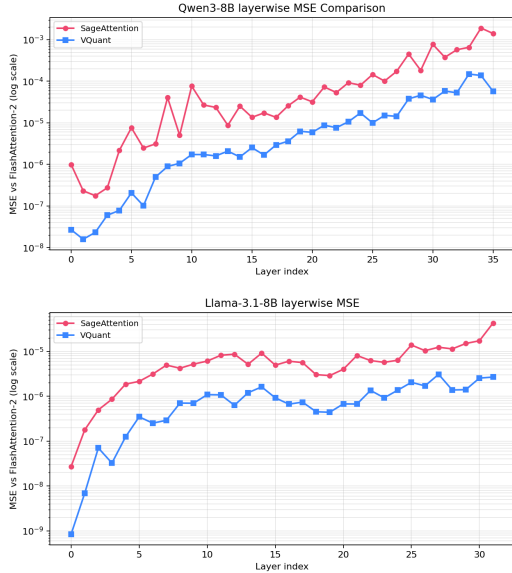


Figure 5: **Layer-wise MSE reduction factor in Prefill**. Using FA2 as the reference, we compute the MSE of the intermediate attention output (input to ϕ_{proj}) per layer. We plot the MSE reduction factor $\text{MSE}_{\text{Sage}}/\text{MSE}_{\text{HyQuant}}$, where larger is better.

Table 6: Throughput (tokens/s) under increasing batch size on Qwen3-8B with a 32K prefix, single H100-80GB. All methods except FA2 are tested under 4-bit quantization for KV cache.

Batch	FA2	HyQuant	KIVI	KVTuner
1	42.8	43.7	23.7	24.2
4	122.2	84.2	32.5	33.4
8	152.7	155.6	34.9	35.6
16	OOM	231.6	OOM	OOM
32	OOM	OOM	OOM	OOM

accuracy–efficiency trade-off with strong decoding efficiency. Across the evaluated benchmarks, HyQuant remains close to the full-precision baseline while avoiding the larger degradation observed in strict low-bit quantization baselines. On some datasets, HyQuant slightly exceeds FA2, which we treat as normal evaluation variance rather than evidence that quantization improves the underlying model capability.

Table 7: **Decode kernel latency under different prefix lengths (ms/token)**. Speedup is computed as FA2 / HyQuant.

Sequence length	FA2	HyQuant	Speedup
1,024	0.224	0.170	1.32×
2,048	0.416	0.173	2.40×
4,096	0.805	0.263	3.06×
8,192	1.604	0.478	3.36×
16,384	3.181	0.903	3.52×
32,768	6.354	1.775	3.58×

Table 8: **End-to-end decode speed relative to FA2 under different prefix lengths**. Values are normalized by the FA2 baseline; values above 1.0× indicate speedup.

Sequence length	KIVI	KVTuner	HyQuant
1,024	0.72×	0.78×	1.04×
2,048	0.72×	0.79×	1.07×
4,096	0.73×	0.80×	1.09×
8,192	0.73×	0.80×	1.12×
16,384	0.71×	0.74×	1.13×
32,768	0.69×	0.72×	1.17×

5.4 Ablations

5.4.1 Importance of the sliding window and vertical-line awareness

To validate the effectiveness of the sliding window and vertical-line awareness design, we conduct ablation studies on Qwen3-8B with LongBench v1. The results show that both the local full-precision window and vertical-line-aware retention reduce the MSE during the Prefill stage.

5.4.2 Vertical-line token ratio (k)

We vary the retained vertical-line token ratio k while keeping other settings fixed to study the accuracy–efficiency trade-off. As shown in Table 11, increasing the top- k ratio generally improves accuracy and reduces quantization error, but also increases the full-precision KV budget. We therefore use top-5% as a practical trade-off unless

otherwise specified.

5.4.3 Local full-precision window size

We vary the local full-precision window size to evaluate sensitivity to local-context dependency and verify its complementarity with vertical-line-aware retention. As shown in Table 10, a larger window size slightly improves accuracy, likely because more recent tokens remain in full precision during inference.

5.5 Additional Cost Overhead

The additional runtime overhead mainly comes from vertical-line identification, including accumulating query vectors, executing a matrix multiplication every 64 tokens, and summing the resulting columns. Our evaluation shows that this additional vertical-line identification overhead accounts for only **3% to 5%** of the total runtime. This minor overhead is largely offset by the hybrid-precision attention kernels and Triton implementations.

For memory overhead, HyQuant buffers at most 64 query vectors in FP16, resulting in an auxiliary memory cost of $64 \times H_Q \times d \times 2$ bytes. Keeping 5% of vertical-line tokens in full precision increases the non-window KV cache size by about 15% compared with strict 4-bit quantization; the total overhead additionally depends on the local-window size. In practice, the improvements in end-to-end speed and accuracy offset this additional memory overhead. Detailed analysis can be seen in B.7.

6 Conclusion

In this paper, we present HyQuant, a hybrid-precision quantization framework for efficient long-context LLM inference. HyQuant identifies vertical-line tokens using accumulated column-wise attention scores, retains these tokens and a local window in full precision, and quantizes the remaining majority to low-bit formats. We further implement fused operators that integrate low-bit computation, full-precision retention, and on-the-fly dequantization for Prefill and Decode execution. Experimental results show that HyQuant achieves $1.32\times$ to $3.58\times$ decode-kernel speedup and $1.04\times$ to $1.17\times$ end-to-end decode speedup while maintaining near-full-precision accuracy across multiple long-context and reasoning benchmarks.

Acknowledgments

This work is sponsored by National Natural Science Foundation of China (No.62602410), Natural Science Foundation of Shanghai (No.24ZR1430600), Tencent Basic Platform Technology Rhino-Bird Focused Research Program and Shanghai Key Laboratory of Trusted Data Circulation and Governance, and Web3.

Limitations

- The effect of retaining critical vertical-line tokens is most visible in long-context tasks. For short-context tasks, HyQuant may not provide the same level of improvement, but it still maintains competitive performance relative to the full-precision baseline. See Appendix B.4, Table 13.
- Our experiments are conducted on NVIDIA H100 GPUs. On high-end GPUs, end-to-end speedups can be less pronounced at short context lengths because the decoding pipeline is not fully memory-bound. Due to GPU memory limitations, we have not evaluated larger-scale models beyond Qwen3-32B, such as Qwen3-80B.
- We have made a broad assessment across tasks and models. However, it is unknown whether our method remains effective in agent / coding-related settings.

References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. [LongBench: A bilingual, multi-task benchmark for long context understanding](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2025. [PyramidKV: Dynamic KV cache compression based on pyramidal information funneling](#). In *Conference on Language Modeling*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman.

2021. [Training verifiers to solve math word problems](#). *Preprint*, arXiv:2110.14168.
- Tri Dao. 2024. [FlashAttention-2: Faster attention with better parallelism and work partitioning](#). In *Proceedings of the International Conference on Learning Representations*.
- DeepSeek-AI. 2025. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *Preprint*, arXiv:2501.12948.
- Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. 2025. [Not all heads matter: A head-level KV cache compression method with integrated retrieval and reasoning](#). In *Proceedings of the International Conference on Learning Representations*.
- Gemini Team. 2025. [Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities](#). *Preprint*, arXiv:2507.06261.
- GLM Team. 2024. [ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools](#). *Preprint*, arXiv:2406.12793.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. [The Llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. 2025. [When attention sink emerges in language models: An empirical view](#). In *International Conference on Learning Representations*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. [Measuring mathematical problem solving with the math dataset](#). *Preprint*, arXiv:2103.03874.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. [KVQuant: Towards 10 million context length LLM inference with KV cache quantization](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. [MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Yiwu Yao, Wulong Liu, Sinno Jialin Pan, and Mingxuan Yuan. 2025. [KV-Tuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference](#). In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 36451–36485. PMLR.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. [SnapKV: LLM knows what you are looking for before generation](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. [Let’s verify step by step](#). *Preprint*, arXiv:2305.20050.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. [KIVI: A tuning-free asymmetric 2bit quantization for KV cache](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344. PMLR.
- OpenAI. 2024. OpenAI o1 System Card. <https://openai.com/index/openai-o1-system-card/>.
- Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. [Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free](#). In *Advances in Neural Information Processing Systems*, volume 38.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. [Efficient streaming language models with attention sinks](#). In *Proceedings of the International Conference on Learning Representations*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.
- Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. 2025. [SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration](#). In *International Conference on Learning Representations*.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. [H₂O: Heavy-hitter](#)

oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36.

Xingyu Zheng, Yuye Li, Haoran Chu, Yue Feng, Xudong Ma, Zining Wang, Jie Luo, Jinyang Guo, Haotong Qin, Michele Magno, and Xianglong Liu. 2026. [An empirical study of Qwen3 quantization](#). *Visual Intelligence*, 4:11.

A Implementation Details and Algorithms

This appendix provides implementation-level details for HyQuant. We include the algorithmic steps that are omitted from the main text due to space constraints. Algorithm 1 describes how HyQuant identifies vertical-line tokens from accumulated attention signals. Algorithms 2 and 3 describe the hybrid-precision attention operators used in the prefill and decode stages. Algorithm 4 gives the cache reorganization procedure at the prefill-to-decode boundary. In the algorithms below, VTI abbreviates VERTICALTOKENIDENTIFICATION.

Algorithm 1 implements the lightweight vertical-line token selection used by HyQuant. The algorithm first excludes the recent local window and then estimates the column-wise importance of the non-window prefix. For GQA models, it computes the scores in a GQA-native form without explicitly materializing repeated KV heads. The returned indices are used as the full-precision vertical-line token set in both prefill and decode.

A.1 Vertical-Line Token Identification

Algorithm 1 Vertical Token Identification

Input: Query $Q \in \mathbb{R}^{B \times H_Q \times L_Q \times D}$
Input: Key $K \in \mathbb{R}^{B \times H \times L_K \times D}$, where $H \in \{H_Q, H_{KV}\}$
Input: Window size W , ratio $r \in (0, 1]$, GQA factor $g = H_Q/H_{KV}$
Output: Vertical key indices $\mathcal{I} \in \mathbb{Z}^{B \times H_Q \times k}$

- 1: $P \leftarrow \max(0, L_K - W)$ ▷ exclude the local window
- 2: **if** $P = 0$ **then**
- 3: **return** $\emptyset$
- 4: **end if**
- 5: $t \leftarrow \min(W, L_Q)$
- 6: $\bar{Q} \leftarrow \frac{1}{t} \sum_{i=L_Q-t+1}^{L_Q} Q_{:,i,:}$ ▷ tail-query proxy
- 7: $K^{\text{pre}} \leftarrow K_{:,1:P,:}$
- 8: **if** $H = H_Q$ **then**
- 9: $S \leftarrow \bar{Q} K^{\text{pre}^\top}$
- 10: **else**
- 11: Reshape $\bar{Q} \rightarrow \bar{Q}^g \in \mathbb{R}^{B \times H_{KV} \times g \times D}$
- 12: $S \leftarrow \text{einsum}("bhrd, bhpd \rightarrow bhrp", \bar{Q}^g, K^{\text{pre}})$
- 13: Reshape $S \rightarrow \mathbb{R}^{B \times H_Q \times P}$
- 14: **end if**
- 15: $k \leftarrow \min(P, \max(1, \lceil P \cdot r \rceil))$
- 16: $\mathcal{I} \leftarrow \text{TOPK}(S, k, \text{dim}=-1, \text{largest})$
- 17: **return** $\mathcal{I}$ ▷ indices in prefix $[0, P)$

A.2 Hybrid-Precision Prefill Operator

Algorithm 2 shows the prefill-stage hybrid-precision attention operator. The key/value sequence is partitioned into three regions: a low-bit quantizable prefix, a full-precision vertical-line region, and a full-precision local window. HyQuant

scans these regions in a segmented FlashAttention-style online softmax, so the quantized and full-precision paths are fused into a single logical operator rather than executed as separate attention calls.

Algorithm 2 HyQuant Prefill: Hybrid-Precision Quantized Attention

Input: Query block Q_b ; full K, V ; window size W ; vertical ratio ρ
Input: Bitwidth $B \in \{\text{INT8}, \text{FP8}\}$; tile size B_N ; scale $\alpha = 1/\sqrt{d}$
Output: Output block O_b

- # Stage 1: identify vertical-line keys and reorder KV
- 1: $\mathcal{K}_{\text{Win}} \leftarrow$ last W keys
- 2: $\mathcal{I}_{\text{VL}} \leftarrow \text{VTI}(Q, K, W, \rho)$ ▷ Alg. 1
- 3: $K, V \leftarrow \text{REORDER}(K, V, \mathcal{I}_{\text{VL}})$ ▷ place vertical-line keys near the window
- 4: Partition $[0, L_K)$ into $\mathcal{K}_Q, \mathcal{K}_{\text{VL}}$, and $\mathcal{K}_{\text{Win}}$
- # Stage 2: quantize the quantizable prefix
- 5: **for** each tile T of size B_N in $\mathcal{K}_Q$ **do**
- 6: $s_T^K \leftarrow \max(|K_T|)/c_B$
- 7: $K_T^Q \leftarrow \text{round}(K_T/s_T^K)$ ▷ $c_B = 127$ for INT8; $c_B = 448$ for FP8
- 8: Compute the query scale s^Q for the corresponding query block
- 9: **end for**
- # Stage 3: segmented online softmax
- 10: $(m, \ell, \text{acc}) \leftarrow (-\infty, 0, \mathbf{0})$
- # Segment A: quantized prefix
- 11: **for** each tile $T \subseteq \mathcal{K}_Q$ **do**
- 12: $P_T \leftarrow (Q_b^Q K_T^{Q^\top})(s^Q s_T^K) \alpha$
- 13: $(m, \ell, \text{acc}) \leftarrow \text{FlashOS}(P_T, V_T; m, \ell, \text{acc})$
- 14: **end for**
- # Segment B: vertical-line tail
- 15: **for** each tile $T \subseteq \mathcal{K}_{\text{VL}}$ **do**
- 16: $P_T^{\text{VL}} \leftarrow Q_b K_T^\top \alpha$
- 17: $(m, \ell, \text{acc}) \leftarrow \text{FlashOS}(P_T^{\text{VL}}, V_T; m, \ell, \text{acc})$
- 18: **end for**
- # Segment C: local full-precision window
- 19: **for** each tile $T \subseteq \mathcal{K}_{\text{Win}}$ **do**
- 20: Apply the per-row causal mask within T
- 21: $P_T^{\text{Win}} \leftarrow Q_b K_T^\top \alpha$
- 22: $(m, \ell, \text{acc}) \leftarrow \text{FlashOS}(P_T^{\text{Win}}, V_T; m, \ell, \text{acc})$
- 23: **end for**
- 24: **return** $O_b \leftarrow \text{acc}/\ell$

A.3 Hybrid-Precision Decode Operator

Algorithm 3 describes the decode-stage attention kernel. Decode is memory-bandwidth dominated, so HyQuant stores most historical KV states in low-bit format and dequantizes them on the fly during attention computation. The full-precision vertical-line tokens, staging buffer, and local window are handled as separate segments and merged through the same online softmax state. This avoids materializing a full-precision KV cache while preserving the most error-sensitive tokens.

Algorithm 3 HyQuant Decode: Hybrid-Precision KV-Cache Attention

Input: Single-token query q_t at step t
Input: KV buffers (K^Q, V^Q) , (K^{VL}, V^{VL}) , (K^S, V^S) , (K^{Win}, V^{Win})
Input: Bitwidths (B_K, B_V) ; tile size B_N ; split count S ; GQA factor g ; scale $\alpha = 1/\sqrt{d}$
Output: Output o_t

- 1: $(m, \ell, acc) \leftarrow (-\infty, 0, \mathbf{0})$
 # Segment 1: quantized prefix
- 2: Partition K^Q into S chunks along the sequence axis
- 3: **for** $s = 1, \dots, S$ **in parallel do**
- 4: Dequantize $\hat{K}_s, \hat{V}_s$ from packed low-bit storage
- 5: Pack g GQA-grouped query heads into $\tilde{q}_t \in \mathbb{R}^{g \times d}$
- 6: $P_s \leftarrow \tilde{q}_t \hat{K}_s^\top \alpha$
- 7: $(m_s, \ell_s, acc_s) \leftarrow \text{FlashOS}(P_s, \hat{V}_s; -\infty, 0, \mathbf{0})$
- 8: **end for**
- 9: $\mathcal{S}_{\text{split}} \leftarrow \{(m_s, \ell_s, acc_s)\}_{s=1}^S$
- 10: $(m, \ell, acc) \leftarrow \text{ReduceSoftmax}(\mathcal{S}_{\text{split}})$
 # Segment 2: vertical-line tokens
- 11: **if** $|K^{VL}| > 0$ **then**
- 12: **for** each tile T in K^{VL} **do**
- 13: $P_T^{VL} \leftarrow q_t K_T^{VL \top} \alpha$
- 14: $(m, \ell, acc) \leftarrow \text{FlashOS}(P_T^{VL}, V_T^{VL}; m, \ell, acc)$
- 15: **end for**
- 16: **end if**
 # Segment 3: staging buffer
- 17: **if** $|K^S| > 0$ **then**
- 18: **for** each tile T in K^S **do**
- 19: $P_T^S \leftarrow q_t K_T^{S \top} \alpha$
- 20: $(m, \ell, acc) \leftarrow \text{FlashOS}(P_T^S, V_T^S; m, \ell, acc)$
- 21: **end for**
- 22: **end if**
 # Segment 4: local full-precision window
- 23: **for** each tile T in K^{Win} **do**
- 24: $P_T^{Win} \leftarrow q_t K_T^{Win \top} \alpha$
- 25: $(m, \ell, acc) \leftarrow \text{FlashOS}(P_T^{Win}, V_T^{Win}; m, \ell, acc)$
- 26: **end for**
- 27: **return** $o_t \leftarrow acc/\ell$

A.4 Prefill-to-Decode KV-Cache Organization

Algorithm 4 describes how HyQuant reorganizes the KV cache when switching from prefill to decode. The procedure fixes the vertical-line token set selected from the prefill tail queries, keeps the recent local window in full precision, and quantizes the remaining non-window prefix into compact low-bit KV buffers. Newly generated tokens are first placed in a small staging buffer before being merged into the hybrid layout. This design avoids recomputing global token importance at every decode step, while keeping the decode kernel structure simple: it scans the quantized prefix, full-precision vertical-line tokens, staging states, and local window as four separate segments.

Algorithm 4 FREEZE: Prefill→Decode Hybrid KV-Cache Organization

Input: Prefill cache $K, V \in \mathbb{R}^{B \times H_{KV} \times N \times D}$
Input: Tail queries Q_{tail} ; window W ; ratio ρ ; bitwidths (B_K, B_V) ; group size G
Output: Hybrid KV buffers for quantized prefix, vertical-line tokens, staging buffer, and local window
 # VTI abbreviates VERTICALTOKENIDENTIFICATION.

- 1: $K^{Win}, V^{Win} \leftarrow K[:, :, -W :, :], V[:, :, -W :, :]$
- 2: $\mathcal{I}_{VL} \leftarrow \text{VTI}(Q_{\text{tail}}, K, W, \rho)$ ▷ computed once and held fixed
- 3: $K^{VL}, V^{VL} \leftarrow K[:, :, \mathcal{I}_{VL}, :], V[:, :, \mathcal{I}_{VL}, :]$
- 4: $\mathcal{I}_Q \leftarrow [0, N - W) \setminus \mathcal{I}_{VL}$
- 5: $K^Q \leftarrow \text{Quant}_{B_K}(K[:, :, \mathcal{I}_Q, :]; G)$
- 6: $V^Q \leftarrow \text{Quant}_{B_V}(V[:, :, \mathcal{I}_Q, :]; G)$
- 7: Initialize empty staging buffer K^S, V^S with capacity W_s
- 8: Free K, V
- 9: **return** all four buffers

B Additional Experimental Results

This appendix reports additional ablations and auxiliary evaluations supporting Section 5. Unless otherwise stated, results use Qwen3-8B with K4V4 quantization and vertical-line-aware full-precision retention.

B.1 Math Reasoning Results

We additionally report math reasoning accuracy on GSM8K and MATH500. These results complement the LongBench evaluation in the main text.

Table 9: Math reasoning accuracy on Qwen3-8B.

Method	GSM8K	MATH500
FA2	95.88	80.14
KIVI	92.48	72.89
KVTuner	93.69	75.83
SageAttention	94.99	77.60
HyQuant	96.52	78.73

B.2 Sensitivity to the Local Full-Precision Window

We vary the number of recent tokens retained in full precision. A larger local window slightly improves accuracy and reduces attention-output MSE.

Table 10: Sensitivity to the local full-precision window size. MSE is reported in units of 10^{-2} ; lower is better.

Window	Acc.	MSE
64	92.54	9.58
128	95.52	8.48
256	95.74	8.21

Table 11: **Sensitivity to the retained vertical-line token ratio.** LongBench subset results on Qwen3-8B under K4V4 quantization.

Top- k	NarrQA	Qasper	MF-en	HotpotQA	Musique	SAMSum	Pretrieve-en	Avg.
2%	24.24	38.69	45.04	47.50	35.25	35.49	96.19	46.06
4%	23.20	40.72	44.47	53.40	43.23	35.59	98.82	48.49
6%	23.85	37.67	53.38	62.65	41.98	36.52	99.26	50.76
8%	23.65	41.84	48.87	57.96	36.53	37.26	95.23	48.76
10%	26.79	45.73	49.61	56.23	45.40	39.61	100.00	51.91

Table 12: MInference-inspired vertical-only ablation on Qwen3-8B. Metric is F1 for narrativeqa, hotpotqa, triviaqa and ROUGE-L for multi_news. Each dataset uses the first 100 test cases.

Task	Metric	FA2	MInf.-Vertical	HyQuant	$\Delta(\text{Mv.} - \text{FA2})$	$\Delta(\text{HQ.} - \text{Mv.})$
narrativeqa	F1	31.49	14.69	31.49	-16.80	+16.80
hotpotqa	F1	62.83	38.55	62.83	-24.28	+24.28
multi_news	ROUGE-L	14.45	13.87	14.52	-0.59	+0.65
triviaqa	F1	89.86	79.20	91.25	-10.66	+12.05
average		49.66	36.58	49.97	-13.08	+13.39

B.3 Sensitivity to the Vertical-Line Token Ratio

Table 11 reports the effect of changing the retained vertical-line token ratio. Increasing the ratio generally improves the average score, but it also increases the number of full-precision KV states. We use top-5% in the main experiments as a practical accuracy–efficiency trade-off.

B.4 Short-Context Evaluation

Although HyQuant is designed for long-context inference, we also evaluate it on short-context benchmarks. HyQuant remains competitive with the full-precision baseline while substantially outperforming strict 4-bit KIVI.

Table 13: **Short-context evaluation on Qwen3-8B.**

Method	C-Eval	MMLU	GSM8K	Avg.
FA2	85.32	86.45	94.89	88.89
KIVI-K8V8	82.83	84.77	94.42	87.34
KIVI-K4V4	75.26	78.34	91.88	81.83
HyQuant	83.23	84.79	95.43	87.82

B.5 Component-Level MSE Ablation

We further isolate the effect of local-window retention and vertical-line retention. The combination of both components gives the lowest layer-wise MSE.

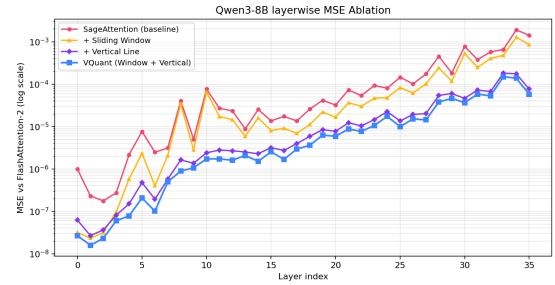


Figure 6: **Component-level MSE ablation of HyQuant.** We compare uniform quantization, local-window retention, vertical-line retention, and their combination on Qwen3-8B. Lower MSE is better.

B.6 Conceptual ablation: marginal value of the quantized long tail

MInference (Jiang et al., 2024) accelerates prefill by computing attention only over dynamically selected sparse positions, including its Vertical-Slash pattern. A vertical-only sparse variant may omit useful long-tail positions in long-context settings, whereas HyQuant retains these positions in low precision. To measure the marginal effect of retaining the long tail, we conduct an ablation study shown in Table 12. The results demonstrate the benefit of keeping all tokens in HyQuant: non-vertical positions remain available in low precision, distinguishing our mixed-precision framework from sparse attention.

B.7 Memory overhead breakdown

Table 14 provides memory overhead analysis for HyQuant.

Component	8K prefix	32K prefix
VL tokens (5% of prefix)	+15.0%	+15.0%
Local window ($W=128$)	+4.7%	+1.2%
Staging buffer ($W_s=128$)	+4.7%	+1.2%
Query buffer (VTI)	< 0.1%	< 0.1%
Total	$\sim 24.4\%$	$\sim 17.4\%$

Table 14: Memory overhead breakdown vs. strict K4V4 (Qwen3-8B, $W=128$, $\rho=5\%$, per layer).

B.8 Experimental Configuration Details

Table 15 summarizes the common experimental configuration used across all methods compared in this paper, including hardware, kernel source, batch size, prefill and generation length, KV bit-width, and memory budget.

Table 15: Experimental configuration used for all compared methods.

Setting	Configuration (all methods)
Hardware	NVIDIA H100 (80GB)
Batch size	1
Prefill length	Up to 128K tokens
Generation length	Up to 32K tokens
KV bit-width	4-bit for all quantization methods
Memory budget	Unconstrained;
Window size	128